Startup Success Prediction

April 27, 2026

[1]:
```
import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt
```

[2]:
```
df = pd.read_csv(r"C:\Users\DELL\OneDrive\Desktop\ML project\startup data.csv",␣
↪encoding='latin1')
```

[3]:
```
print(type(df))
print(df.shape)
df.head()
```

```
<class 'pandas.core.frame.DataFrame'>
(923, 49)
```

[3]:

	Unnamed: 0	state_code	latitude	longitude	zip_code	id	
0	1005	CA	42.358880	-71.056820	92101	c:6669	
1	204	CA	37.238916	-121.973718	95032	c:16283	
2	1001	CA	32.901049	-117.192656	92121	c:65620	
3	738	CA	37.320309	-122.050040	95014	c:42668	
4	1002	CA	37.779281	-122.419236	94105	c:65806	

	city	Unnamed: 6	name	labels	…	\
0	San Diego	NaN	Bandsintown	1	…	
1	Los Gatos	NaN	TriCipher	1	…	
2	San Diego	San Diego CA 92121	Plixi	1	…	
3	Cupertino	Cupertino CA 95014	Solidcore Systems	1	…	
4	San Francisco	San Francisco CA 94105	Inhale Digital	0	…	

	object_id	has_VC	has_angel	has_roundA	has_roundB	has_roundC	has_roundD	\
0	c:6669	0	1	0	0	0	0	
1	c:16283	1	0	0	1	1	1	
2	c:65620	0	0	1	0	0	0	
3	c:42668	0	0	0	1	1	1	
4	c:65806	1	1	0	0	0	0	

	avg_participants	is_top500	status
0	1.0000	0	acquired
1	4.7500	1	acquired

```
2            4.0000          1  acquired
3            3.3333          1  acquired
4            1.0000          1    closed

[5 rows x 49 columns]
```

[4]:
```
df.columns = df.columns.str.strip()
print(df.columns.tolist())
```

```
['Unnamed: 0', 'state_code', 'latitude', 'longitude', 'zip_code', 'id', 'city',
'Unnamed: 6', 'name', 'labels', 'founded_at', 'closed_at', 'first_funding_at',
'last_funding_at', 'age_first_funding_year', 'age_last_funding_year',
'age_first_milestone_year', 'age_last_milestone_year', 'relationships',
'funding_rounds', 'funding_total_usd', 'milestones', 'state_code.1', 'is_CA',
'is_NY', 'is_MA', 'is_TX', 'is_otherstate', 'category_code', 'is_software',
'is_web', 'is_mobile', 'is_enterprise', 'is_advertising', 'is_gamesvideo',
'is_ecommerce', 'is_biotech', 'is_consulting', 'is_othercategory', 'object_id',
'has_VC', 'has_angel', 'has_roundA', 'has_roundB', 'has_roundC', 'has_roundD',
'avg_participants', 'is_top500', 'status']
```

[5]:
```
df = df.loc[:, ~df.columns.str.contains('^Unnamed')]
```

[12]:
```
print(df.columns.tolist())
```

```
['state_code', 'latitude', 'longitude', 'zip_code', 'id', 'city', 'name',
'status', 'founded_at', 'closed_at', 'first_funding_at', 'last_funding_at',
'age_first_funding_year', 'age_last_funding_year', 'age_first_milestone_year',
'age_last_milestone_year', 'relationships', 'funding_rounds',
'funding_total_usd', 'milestones', 'state_code.1', 'is_CA', 'is_NY', 'is_MA',
'is_TX', 'is_otherstate', 'category_code', 'is_software', 'is_web', 'is_mobile',
'is_enterprise', 'is_advertising', 'is_gamesvideo', 'is_ecommerce',
'is_biotech', 'is_consulting', 'is_othercategory', 'object_id', 'has_VC',
'has_angel', 'has_roundA', 'has_roundB', 'has_roundC', 'has_roundD',
'avg_participants', 'is_top500', 'status']
```

[13]:
```
df = df.loc[:, ~df.columns.duplicated()]
```

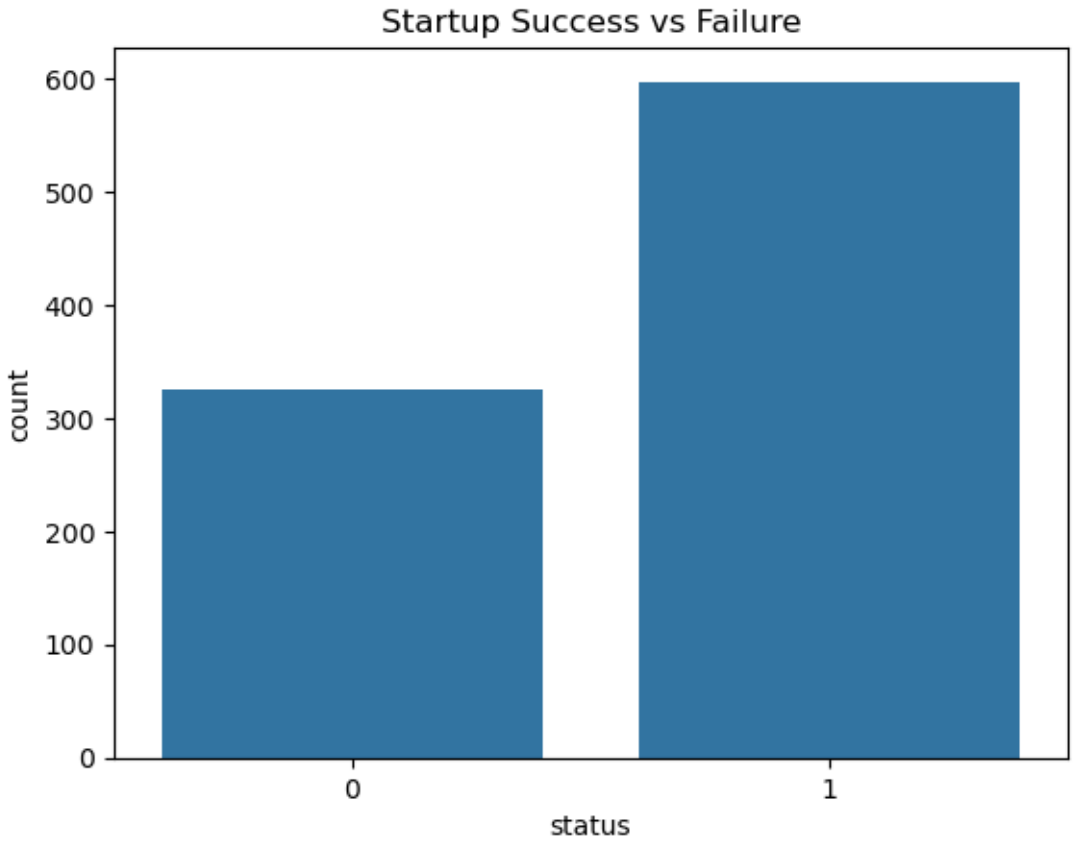
[14]:
```
print(df.columns.tolist())
```

```
['state_code', 'latitude', 'longitude', 'zip_code', 'id', 'city', 'name',
'status', 'founded_at', 'closed_at', 'first_funding_at', 'last_funding_at',
'age_first_funding_year', 'age_last_funding_year', 'age_first_milestone_year',
'age_last_milestone_year', 'relationships', 'funding_rounds',
'funding_total_usd', 'milestones', 'state_code.1', 'is_CA', 'is_NY', 'is_MA',
'is_TX', 'is_otherstate', 'category_code', 'is_software', 'is_web', 'is_mobile',
'is_enterprise', 'is_advertising', 'is_gamesvideo', 'is_ecommerce',
'is_biotech', 'is_consulting', 'is_othercategory', 'object_id', 'has_VC',
'has_angel', 'has_roundA', 'has_roundB', 'has_roundC', 'has_roundD',
'avg_participants', 'is_top500']
```

[15]:
```
print(df['status'].value_counts())
```

```
status
1    597
0    326
Name: count, dtype: int64
```

[16]:
```
sns.countplot(x=df['status'])
plt.title("Startup Success vs Failure")
plt.show()
```



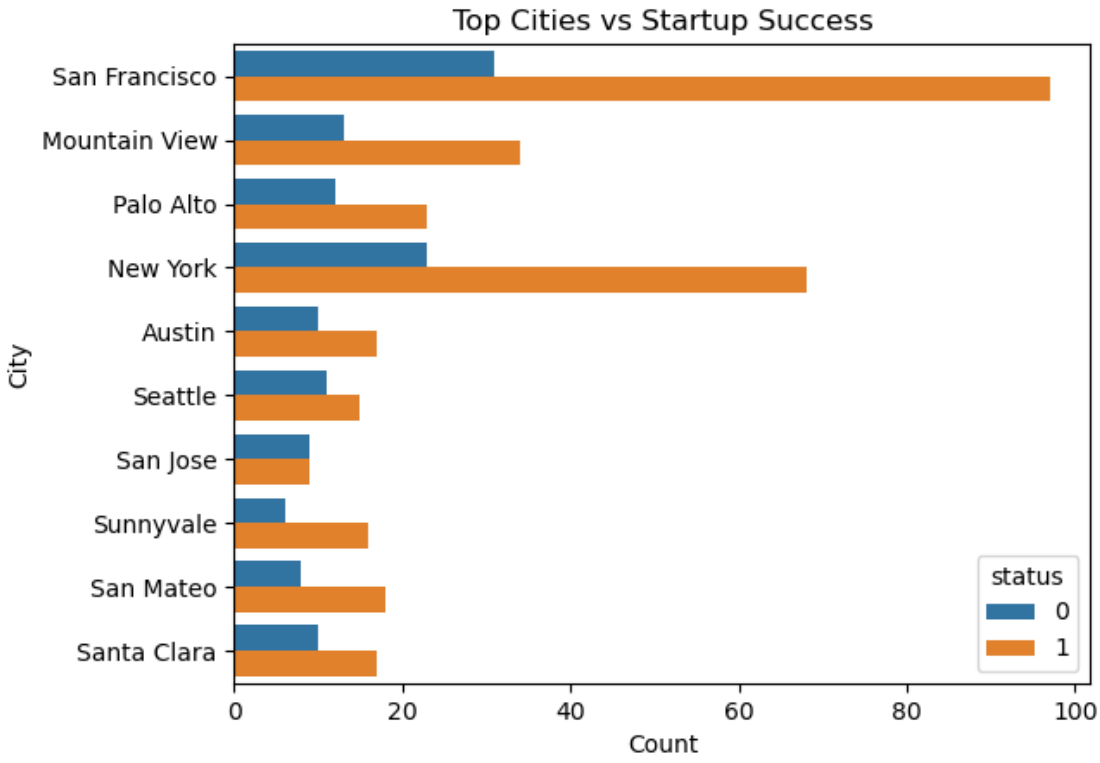
[17]:
```
print('city' in df.columns)
```

```
True
```

[18]:
```
df['city'] = df['city'].fillna('Unknown')

top_cities = df['city'].value_counts().head(10).index

sns.countplot(
    y=df.loc[df['city'].isin(top_cities), 'city'],
```

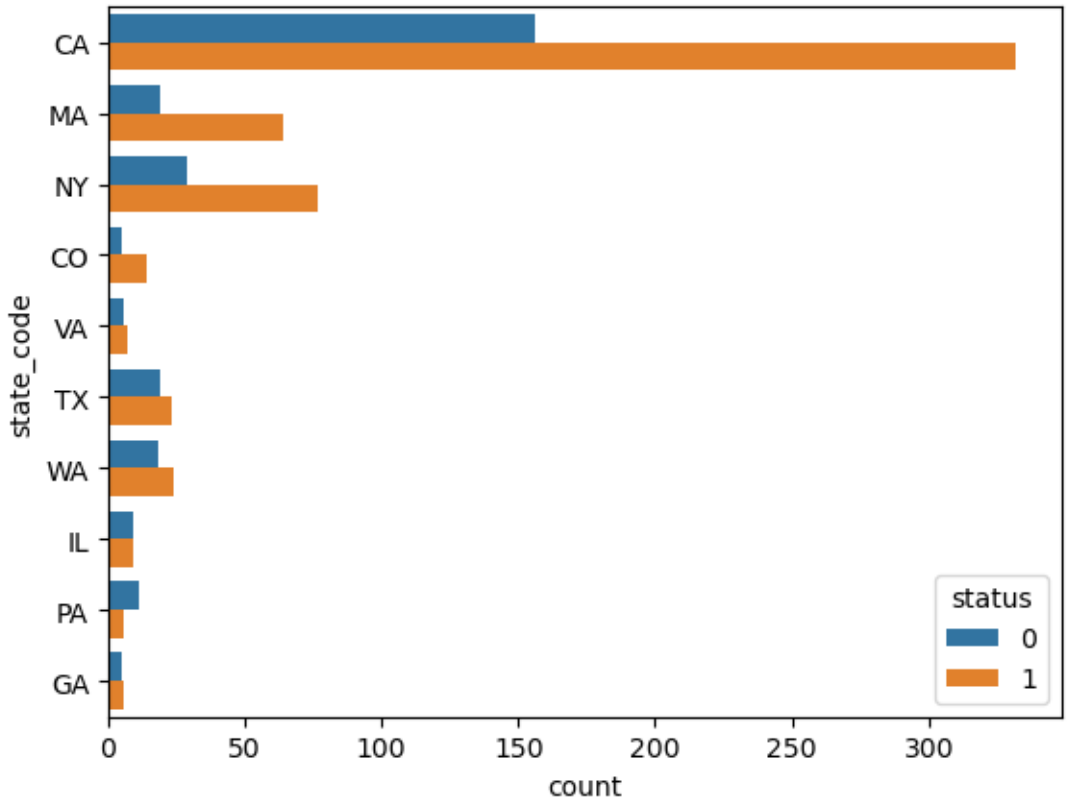
```
    hue=df.loc[df['city'].isin(top_cities), 'status']
)

plt.title("Top Cities vs Startup Success")
plt.xlabel("Count")
plt.ylabel("City")
plt.show()
```

```
C:\Users\DELL\AppData\Local\Temp\ipykernel_20556\3472442930.py:1:
SettingWithCopyWarning:
A value is trying to be set on a copy of a slice from a DataFrame.
Try using .loc[row_indexer,col_indexer] = value instead

See the caveats in the documentation: https://pandas.pydata.org/pandas-
docs/stable/user_guide/indexing.html#returning-a-view-versus-a-copy
  df['city'] = df['city'].fillna('Unknown')
```



[19]:
```
top_states = df['state_code'].value_counts().head(10).index

sns.countplot(
    y=df.loc[df['state_code'].isin(top_states), 'state_code'],
    hue=df.loc[df['state_code'].isin(top_states), 'status']
```

```
)
plt.show()
```



[20]:
```
print('latitude' in df.columns, 'longitude' in df.columns)
```

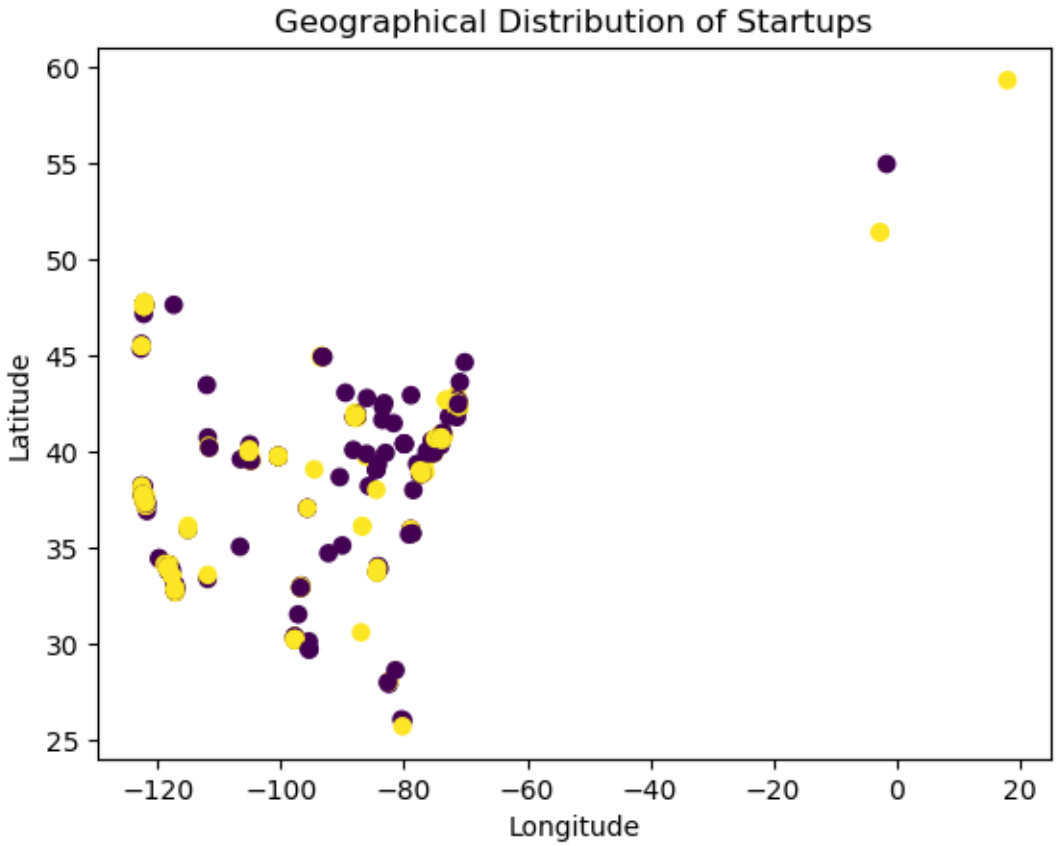
```
True True
```

[21]:
```
df['latitude'] = pd.to_numeric(df['latitude'], errors='coerce')
df['longitude'] = pd.to_numeric(df['longitude'], errors='coerce')

df = df.dropna(subset=['latitude', 'longitude'])
```

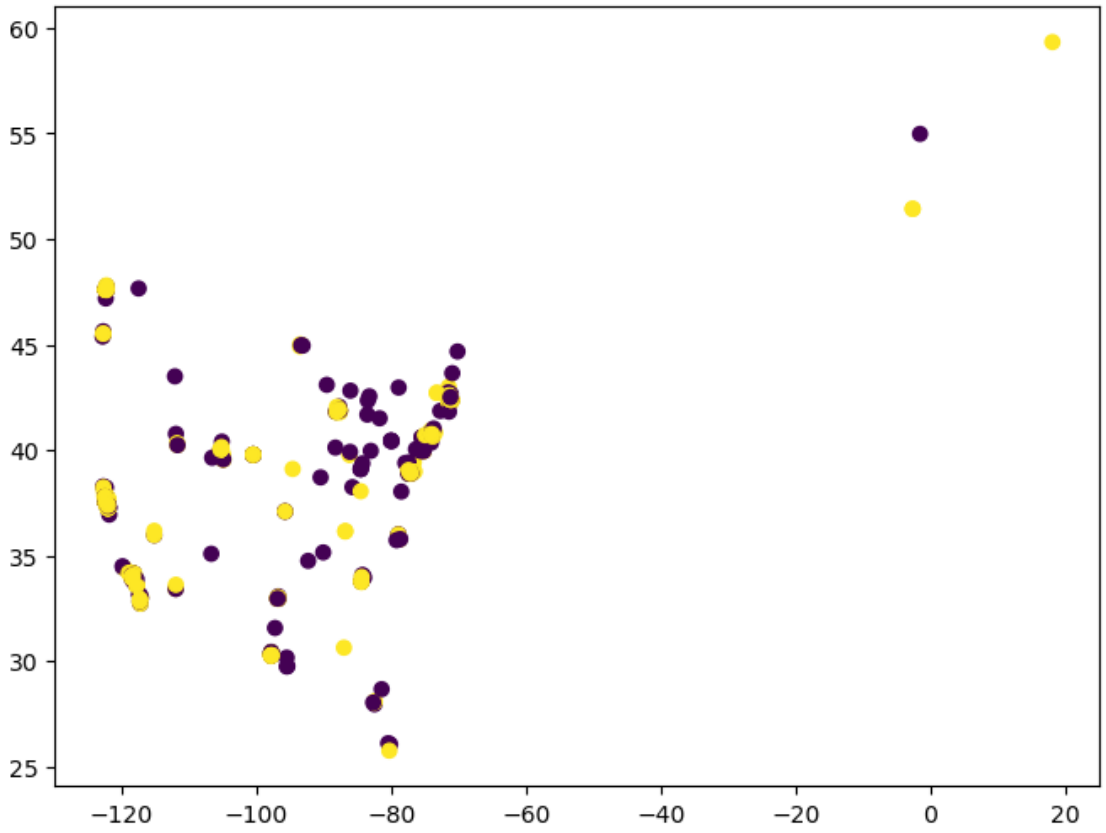
```
C:\Users\DELL\AppData\Local\Temp\ipykernel_20556\3298397388.py:1:
SettingWithCopyWarning:
A value is trying to be set on a copy of a slice from a DataFrame.
Try using .loc[row_indexer,col_indexer] = value instead

See the caveats in the documentation: https://pandas.pydata.org/pandas-
docs/stable/user_guide/indexing.html#returning-a-view-versus-a-copy
  df['latitude'] = pd.to_numeric(df['latitude'], errors='coerce')
C:\Users\DELL\AppData\Local\Temp\ipykernel_20556\3298397388.py:2:
SettingWithCopyWarning:
```

```
A value is trying to be set on a copy of a slice from a DataFrame.
Try using .loc[row_indexer,col_indexer] = value instead

See the caveats in the documentation: https://pandas.pydata.org/pandas-
docs/stable/user_guide/indexing.html#returning-a-view-versus-a-copy
  df['longitude'] = pd.to_numeric(df['longitude'], errors='coerce')
```

[22]:
```
plt.scatter(df['longitude'], df['latitude'], c=df['status'])
plt.xlabel("Longitude")
plt.ylabel("Latitude")
plt.title("Geographical Distribution of Startups")
plt.show()
```



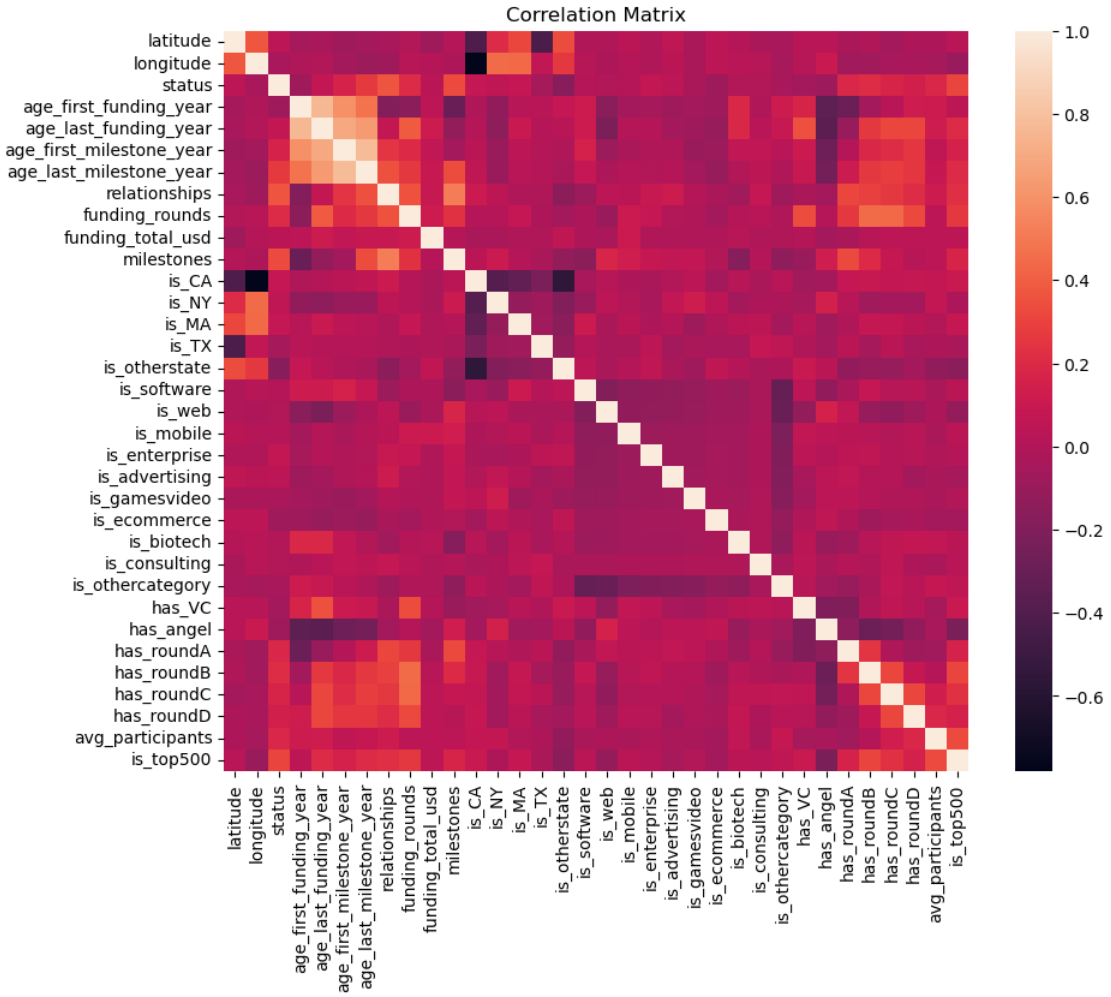
[23]:
```
plt.figure(figsize=(8,6))
plt.scatter(df['longitude'], df['latitude'], c=df['status'])
plt.show()
```



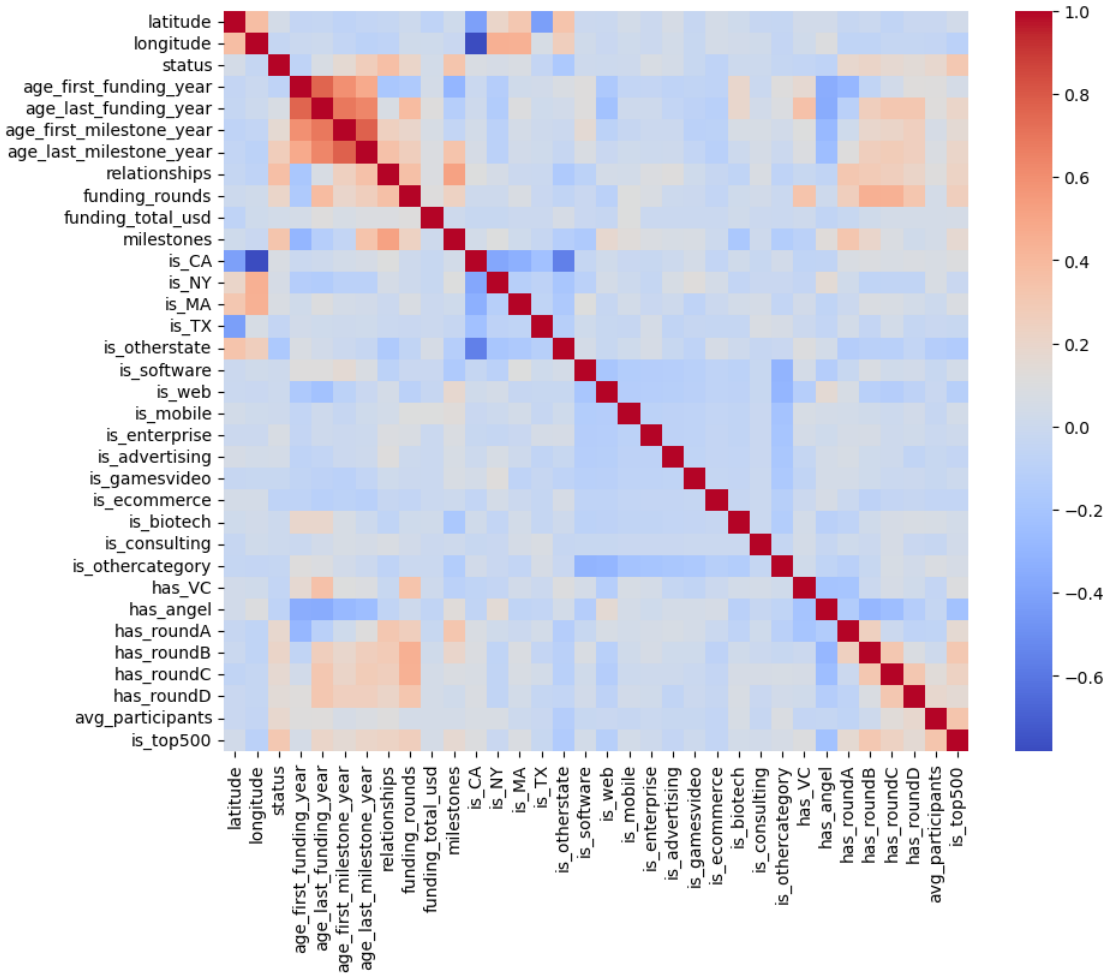
[24]:
```
numeric_df = df.select_dtypes(include=['int64', 'float64'])
```

[25]:
```
corr = numeric_df.corr()
```

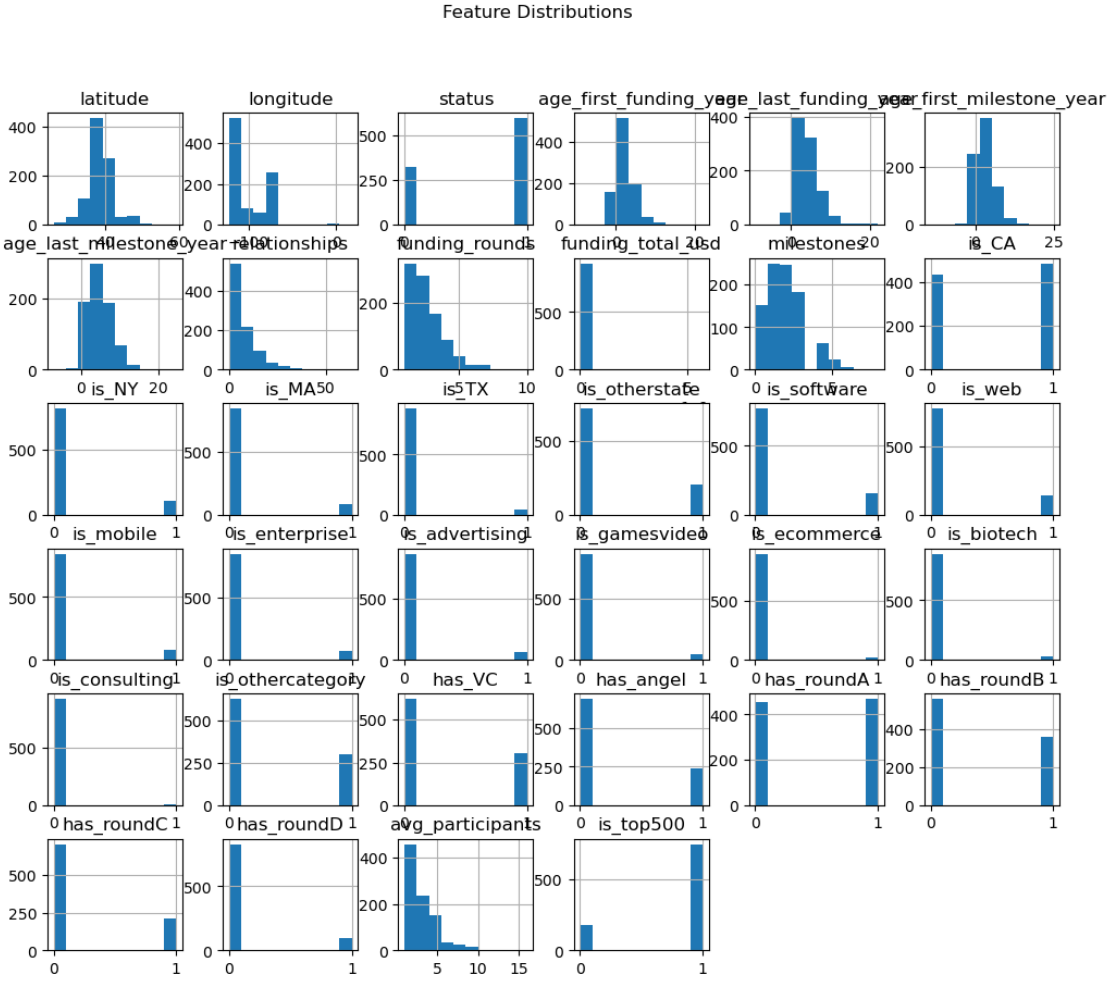
[26]:
```
import seaborn as sns
import matplotlib.pyplot as plt

plt.figure(figsize=(10,8))
sns.heatmap(corr)
plt.title("Correlation Matrix")
plt.show()
```



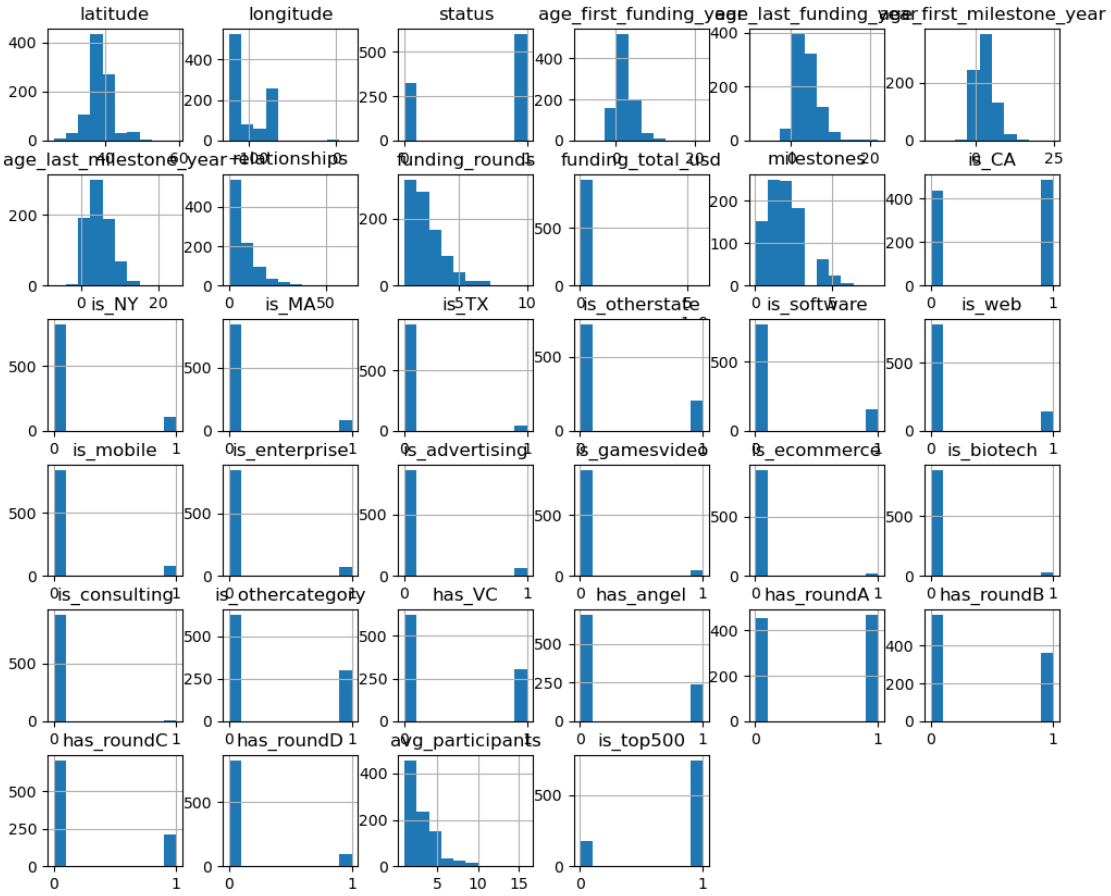
[27]:
```
plt.figure(figsize=(10,8))
sns.heatmap(corr, annot=False, cmap='coolwarm')
plt.show()
```



[28]:
```
df.hist(figsize=(12,10))
plt.suptitle("Feature Distributions")
plt.show()
```



[29]:
```
numeric_df = df.select_dtypes(include=['int64', 'float64'])

numeric_df.hist(figsize=(12,10))
plt.show()
```



[30]:
```
# Select only numeric columns
df_ml = df.select_dtypes(include=['int64', 'float64'])

# Drop rows with missing values
df_ml = df_ml.dropna()
```

[31]:
```
X = df_ml.drop('status', axis=1)
y = df_ml['status']
```

[32]:
```
from sklearn.model_selection import train_test_split

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)
```

[33]:
```
from sklearn.linear_model import LogisticRegression

model = LogisticRegression(max_iter=1000)
model.fit(X_train, y_train)
```

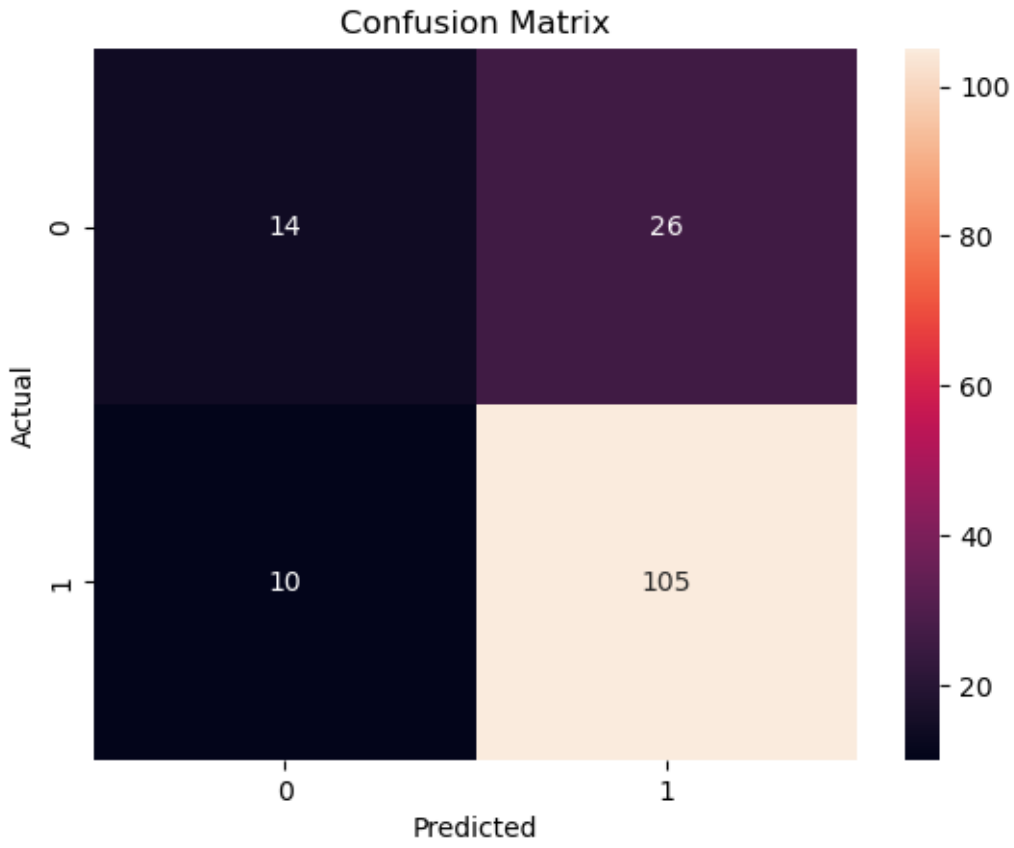
[33]:
```
LogisticRegression(max_iter=1000)
```

[34]:
```
y_pred = model.predict(X_test)
```

[35]:
```
from sklearn.metrics import accuracy_score, confusion_matrix,␣
 ↪classification_report

print("Accuracy:", accuracy_score(y_test, y_pred))
print("\nConfusion Matrix:\n", confusion_matrix(y_test, y_pred))
print("\nReport:\n", classification_report(y_test, y_pred))
```

```
Accuracy: 0.7677419354838709

Confusion Matrix:
 [[ 14  26]
 [ 10 105]]

Report:
              precision    recall  f1-score   support

           0       0.58      0.35      0.44        40
           1       0.80      0.91      0.85       115

    accuracy                           0.77       155
   macro avg       0.69      0.63      0.65       155
weighted avg       0.75      0.77      0.75       155
```

[36]:
```
import seaborn as sns

cm = confusion_matrix(y_test, y_pred)

sns.heatmap(cm, annot=True, fmt='d')
plt.title("Confusion Matrix")
plt.xlabel("Predicted")
plt.ylabel("Actual")
plt.show()
```



[37]:
```
from sklearn.ensemble import RandomForestClassifier

rf = RandomForestClassifier()
rf.fit(X_train, y_train)

y_pred_rf = rf.predict(X_test)

print("RF Accuracy:", accuracy_score(y_test, y_pred_rf))
```

```
RF Accuracy: 0.7741935483870968
```

[ ]: